BCSE306L Artificial Intelligence
Course Project

Course-Eligibility Reasoner

Module & Topic: Knowledge Representation and Reasoning
Rule-based Inference (Forward and Backward Chaining with First-Order Logic unification)

Submitted By: Saatvika K - 24BCE0929
Deepak James - 24BCE0923
Mohammed Wasif - 24BCE2523

Submitted To: NANCY LYDIA R

Code repository: https://github.com/Saatvika04/Course_Eligibility_Reasoner.git

Problem Statement and Scope

The problem:

A university student wants to know whether they may register for a given course. Eligibility is not a simple "have you passed this one course" question: a course may require several prerequisites that must all be satisfied (AND), may accept any one of several alternatives (OR), may allow an administrative waiver in place of a prerequisite, and may require a corequisite to be taken in the same term.
These constraints can also chain (a prerequisite may itself have prerequisites) so the answer can depend on a multi-step chain of reasoning rather than a single lookup.

Why it matters:

Hand-checking eligibility against a course catalogue is error-prone and scales poorly, and the rules themselves (AND/OR nesting, waivers that apply to one specific course only, corequisites) are subtle enough that a small ad-hoc script easily gets them wrong. This project turns the eligibility problem into a symbolic reasoning task: it encodes the catalogue and student record as a formal knowledge base and answers queries with two independent inference algorithms that must agree. Because the reasoning is rule-based rather than statistical, every "yes" or "no" can be explained in plain English and traced step by step — a requirement for anything a registrar would trust.

Inputs:

(1) A knowledge base describing courses, each with a prerequisite expression over AND/OR and a list of corequisites. Students, each with completed courses, courses currently being taken, and waivers.
(2) A query: a student id and a target course id.

Outputs:

An eligibility verdict (eligible / not eligible),
the missing prerequisites and corequisites,
a step-by-step reasoning trace, and
a plain-English explanation of the verdict.

Scope and boundaries:

- Eligibility is decided symbolically; no machine learning is used, and no external inference library is used. The inference engine is implemented from scratch. Eligibility is deliberately distinct from completion. Being eligible for a course does not mean the course has been completed; therefore eligibility for a prerequisite never satisfies that prerequisite. A prerequisite is satisfied only by a recorded completed course or a scoped waiver.
- A corequisite is satisfied only if the student is currently taking it.
- Waivers are scoped to an exact (target course, prerequisite) pair; a waiver for one course does not transfer to another.
- The problem is restricted to definite-clause (Horn) knowledge so that both forward and backward chaining terminate and are decidable. Cyclic prerequisites are detected and reported rather than silently looping.

AI Formulation

The problem is formulated as knowledge representation and reasoning over a first order logic (FOL) knowledge base, restricted to definite clauses (Horn clauses), which is the standard setting for both forward and backward chaining (Russell & Norvig, Ch. 9).

Predicate vocabulary:

The domain is compiled into ground facts and Horn-clause rules over the following predicates:

Predicate	Meaning
`completed(S, C)`	student `S` has completed course `C`
`taking(S, C)`	student `S` is currently taking course `C`
`waiver(S, C, P)`	student `S` has a waiver for prerequisite `P` of course `C`
`prereq(C, P)`	`P` is a prerequisite of `C`
`no_prereq(C)`	course `C` has no prerequisites
`coreq(C, Q)`	`Q` is a corequisite of `C`
`prereq_satisfied(S, C, P)`	`S` has satisfied prerequisite `P` for course `C`
`coreqs_satisfied(S, C)`	`S` has satisfied every corequisite of `C`
`eligible(S, C)`	`S` is eligible to take course `C`

Rules (Horn clauses):

The eligible predicate is the goal; the remaining predicates encode the catalogue semantics:

```
prereq_satisfied(S, C, P) :- waiver(S, C, P).        % a waiver satisfies the prereq
prereq_satisfied(S, C, P) :- completed(S, P).        % or completion satisfies it

coreqs_satisfied(S, C) :- taking(S, Q1), ..., taking(S, Qk).
                                                     % every corequisite must be taken

eligible(S, C) :- no_prereq(C), coreqs_satisfied(S, C).
eligible(S, C) :- prereq(C, P), prereq_satisfied(S, C, P), coreqs_satisfied(S, C).
eligible(S, C) :- prereq_satisfied(S, C, P1), ..., prereq_satisfied(S, C, Pn),
                  coreqs_satisfied(S, C).
                                                     % one clause per DNF clause
```

Compiling AND/OR prerequisites to DNF:

A prerequisite expression over and / or / leaves (e.g. {"or": [{"and": ["CS101","MATH101"]}, "CS102"]}) is compiled to disjunctive normal form (DNF). Each DNF clause becomes one eligibility rule whose body requires every leaf in the clause to be prereq_satisfied; multiple clauses become multiple alternative rules. This makes the AND/OR semantics explicit in the rule set and lets both chaining algorithms share one uniform encoding.

Reasoning semantics:

These are the design decisions that make the reasoner correct:

1. Eligibility ≠ completion. The rules derive eligible, never completed. A course reached through a chain is eligible for the next course only if it is actually completed. Being merely eligible for CS203 does not satisfy CS301's prerequisite on CS203.
2. Prerequisites are satisfied by completion or a scoped waiver — never by "eligibility."
3. Corequisites are satisfied by taking only.
4. Waivers are scoped to the exact (course, prerequisite) pair; the fact waiver(S, CS202, MATH101) satisfies prereq_satisfied(S, CS202, MATH101) and nothing else.

Algorithm and Implementation

Two independent inference algorithms are implemented from scratch over the shared fact/rule encoding. Both return the same stable result dictionary {eligible, student_id, course_id, missing_prerequisites, missing_corequisites, trace, inference_steps, rules_examined, derived_facts} .

Unification

Both algorithms rely on a first-order unification routine with an occurs-check (prevents a variable from unifying with a term that contains it, e.g. X with f(X)), which guarantees sound substitution and terminates cyclic structure. Rules are standardized apart before each use (variables are renamed with a fresh serial) so that repeated rule application never captures the wrong variable.

Backward chaining (goal-directed)

Backward chaining starts from the goal eligible(S, C) and works backwards, trying to prove the goal against known facts and rule heads. It is depth-first with an active-path cycle guard: if a goal re-appears on the current proof branch, it is rejected as a cycle rather than recursing forever.

```
SOLVE(goal, bindings):
    goal = ground(goal, bindings)
    if goal in active_path: mark CYCLE; return []
    answers = [σ for each fact if σ = unify(goal, fact, bindings)]
    push goal onto active_path
    for each rule:
        candidate = standardise_apart(rule)
        σ = unify(goal, candidate.head, bindings)
        if σ fails: continue
        partials = [σ]
        for body_goal in candidate.body:
            partials = [s' for s in partials for s' in SOLVE(body_goal, s)]
            if partials empty: break
        append partials to answers
    pop active_path
    if answers empty: record goal as FAILED
    return answers
```

Result interpretation:

eligible is true if any proof is found. If no proof is found, the set of failed sub-goals is inspected: failed prereq_satisfied(S, C, P) goals yield missing prerequisites, and failed taking(S, Q) goals yield missing corequisites. The inference-steps counter counts goals entered (including cycle-rejected ones); rules-examined counts candidate rule heads inspected. No closure is built, so derived_facts is None /N/A.

Forward chaining (data-driven)

Forward chaining starts from the known facts and repeatedly applies rules until no new fact can be derived (a least fixed point). It then checks whether the target eligible(S, C) is in the closure.

```
known = initial facts
loop:
    additions = []
    for rule in rules:
        candidate = standardise_apart(rule)
        partials = [{}]
        for body_goal in candidate.body:          # match body against known facts
            partials = [σ | unify(body_goal, fact, p) for p in partials for fact in known]
            if partials empty: break
        for binding in partials:
            ground any unbound head variables over the KB's finite constants
            derived = ground(candidate.head, binding)
            if derived not in known: additions += derived
    if additions empty: break                      # fixpoint reached
    known += additions
```

Result interpretation:

Because rules may contain head variables that do not appear in the body (e.g. the global completion rule prereq_satisfied(S, C, P) :- completed(S, P) has free C), unbound head variables are range-restricted over the finite constants in the current KB , this prevents deriving irrelevant cross-domain tuples. Inference-steps counts body-to-fact unification attempts; rules-examined counts rule applications visited across closure passes; derived-facts counts unique facts added beyond the input.

Data structures

• Facts: tuples of constant strings, e.g. ("completed", "bob", "CS101").
• Rules: dictionaries {"head": (…), "body": [(…), …]}; variables are Var objects that unify and hash by name.
• Substitutions: dictionaries mapping Var → term, applied recursively.
• Cycle guard: a stack (list) of goals active on the current branch.
• Knowledge base: a validated JSON document {courses: {…}, students: {…}}.

What the team implemented vs. what libraries provide

Component	Implemented by the team	Third-party
KB schema, loader, validator, cycle detection	✔ (from scratch)	—
AND/OR → DNF compilation to facts + Horn rules	✔	—
Unification (with occurs-check)	✔	—
Forward chaining engine	✔	—
Backward chaining engine (with cycle guard)	✔	—
Missing-condition extraction & result shape	✔	—
Labelled evaluation + benchmark harness	✔	—
Plain-English explanation layer	✔	—
CLI / web UI	✔	`streamlit` (UI only)
Plots	✔	`matplotlib` (rendering only)
Tests	✔	`pytest` (runner only)

The entire inference core — unification, both chaining algorithms, and cycle detection — is written from scratch; no inference or logic library is used. Third-party packages only render the UI and plots.

Experiments and Results

Correctness: labelled test cases

An independent labelled suite of 21 cases (T01–T21) covers the full semantics: no prerequisite, single prerequisite, AND (all/missing), OR (one/both branches), corequisite (taking/missing), scoped waiver (valid/non-transferable), prerequisite chains, eligibility-vs completion, a cyclic prerequisite component, unseen student/course, unknown course, OR-of AND, and prerequisite+corequisite combinations. Each case is evaluated by both engines and must match the independently-written expected verdict.

Result: Forward chaining 21/21 correct and backward chaining 21/21 correct (100% each).

Case	Scenario	Expected
T01	course without prerequisites	eligible
T02	single prerequisite satisfied	eligible
T03	AND prerequisites all satisfied	eligible
T04	AND prerequisite missing	not eligible
T05	OR satisfied by one branch	eligible
T06	both OR branches missing	not eligible
T07	corequisite being taken	eligible
T08	corequisite not being taken	not eligible
T09	target-specific waiver applies	eligible
T10	waiver does not transfer to another course	not eligible
T11	middle course in a prerequisite chain	eligible
T12	eligibility is not completion	not eligible
T13	completed intermediate prerequisite	eligible
T14	cyclic prerequisite, no completed dependency	not eligible
T15	unseen student/course, no prerequisites	eligible
T16	unknown course query	not eligible
T17	OR-of-AND satisfied by the AND branch	eligible
T18	OR-of-AND satisfied by the single branch	eligible
T19	OR-of-AND no branch satisfied	not eligible
T20	prerequisite + corequisite both satisfied	eligible
T21	prerequisite satisfied, corequisite missing	not eligible

Sample inputs and outputs

The CLI prints a verdict, missing conditions, counters, and a trace for both algorithms.

```
$ python cli.py carol DS301
Course eligibility: ELIGIBLE
Student: carol    Course: DS301
Missing prerequisites (engine result): None reported
Missing corequisites (not currently taken): None
...
```

carol is eligible for DS301 because she has completed the chain DS101 → DS201 → DS301. The explanation layer renders this:

```
carol is ELIGIBLE for DS301.
  Prerequisite DS201: satisfied.
```

```
$ python cli.py dave CS301
Course eligibility: NOT ELIGIBLE
Missing prerequisites (engine result): CS203
...
```

dave has no completed courses, so CS301's prerequisite CS203 is missing:

```
dave is NOT ELIGIBLE for CS301.
  Prerequisite CS203: MISSING (not completed, no waiver).
```

Comparison: forward vs. backward chaining

Both algorithms were benchmarked on identical synthetic knowledge bases with exactly 50, 100, 250, and 500 predicate rules, three timed runs each, fixed seed. This is the required head-to-head comparison.

Rules	Courses	Algorithm	Median time (ms)	Inference steps	Rules examined	Derived facts
50	24	Backward	1.25	3	150	N/A
50	24	Forward	119.83	55,881	150	412
100	49	Backward	4.30	5	500	N/A
100	49	Forward	633.59	260,284	300	936
250	124	Backward	11.41	5	1,250	N/A
250	124	Forward	5,566.34	1,849,690	750	3,204
500	249	Backward	11.94	3	1,500	N/A
500	249	Forward	11,032.22	5,821,726	1,500	5,208

The same data is plotted in results/benchmark_runtime.png, benchmark_steps.png, and benchmark_rules_examined.png.

Interpretation:

Backward chaining is orders of magnitude faster for a single query because it is goal-directed: it proves only eligible(S, C) and explores only the rules on the path to that goal, so its step count stays tiny as the KB grows. Forward chaining computes the entire closure — every fact derivable from every rule — so its cost grows steeply with rule count (its step count is dominated by body-to-fact unification attempts over the full KB). This is the classic trade-off: backward chaining is the right choice for a single interactive query, while forward chaining would pay off if many queries were answered against one fixed KB (the closure is computed once and reused). The counters measure different operations, so they are compared within an algorithm, not as equal units across the two.

Discussion and Conclusion

Correctness:

The 21-case labelled suite passes against both independent engines, which cross-validates the two very different implementations against one shared semantics. The subtle rules (eligibility ≠ completion, scoped waivers, taking-only corequisites, cycle detection) are each exercised by a dedicated case and held correctly by both engines.

Limitations:

- The labelled suite is a focused functional test, not a proof of correctness for every possible knowledge base.
- The benchmark uses one fixed seed, one student and one target per size, and controlled single-branch prerequisites (so the expanded rule counts are exact); it does not establish that one algorithm is always faster for every query or KB shape. Runtime depends on machine, Python version, and background load; the numbers above are from one machine and are not universal guarantees.
- The KB is restricted to definite (Horn) clauses, so negation and general disjunction in the head are outside scope.

Failure cases handled:

Unknown course or unknown student are answered as "not eligible" with an explanatory note. Prerequisite cycles are detected structurally (and reported as a KB diagnostic) and are also caught at proof time by the backward engine's active-path guard, so cyclic input cannot cause non-termination.

Possible improvements.

(1) A memoised/cached backward prover to avoid re-proving shared sub-goals;
(2) incremental re-evaluation of the forward closure when facts change;
(3) support for temporal constraints (e.g. "within the last N semesters") and numeric conditions such as minimum grade or credit thresholds;
(4) a course-planning view that suggests the cheapest missing prerequisite path.

Conclusion:

The project demonstrates a complete, from-scratch symbolic reasoning system: a formal first-order knowledge base, DNF compilation, unification, and two independent chaining algorithms that agree on 21/21 labelled cases and are compared head-to-head on a scaling benchmark. Every verdict is traceable and explainable, which is the property that makes rule based reasoning suitable for a real registration decision.

Team Contributions and References

Contributions:

Member	Contribution
Saatvika Shree K (24BCE0929)	Knowledge-base schema, generator, loader/validator and structural cycle detection; AND/OR→DNF predicate compilation; plain-English explanation layer; Streamlit web UI; report assembly.
Deepak James (24BCE0923)	First-order unification (with occurs-check); forward-chaining and backward-chaining engines; cycle guard; result shape and missing-condition extraction.
Mohammed Wasif (24BCE2523)	Independent labelled evaluation cases; evaluation and benchmark harness; metrics/normalization and timing; benchmark plot generation; CLI presentation.

References:

1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021 — Ch. 9 (Inference in First-Order Logic; forward and backward chaining).
2. Streamlit documentation — https://docs.streamlit.io
3. Matplotlib documentation — https://matplotlib.org
4. Project source code and test data — https://github.com/Saatvika04/Course_Eligibility_Reasoner.git